COMMODITY RISK PLATFORM

System and Developer Guide

A detailed map of the data, analytics, dashboard, backtesting, database, and operating workflow

PREPARED	21 October 2026
SCOPE	Code-grounded guide for the current repository

Portfolio analytics are illustrative research estimates, not live positions or investment advice.

CONTENTS

A practical reference for understanding, running, validating, and extending the current project.

01 Purpose and operating boundaries

02 End-to-end architecture

03 Repository map

04 Setup and runbook

05 Price acquisition

06 Price cleaning and validation

07 Return construction and calendar alignment

08 Portfolio exposure and P&L

09 Historical VaR and Expected Shortfall

10 Rolling VaR backtest

11 Confidence controls and the 95% versus 99% issue

12 Statistical backtesting tests

13 Dashboard walkthrough

14 Command-line reports and output files

15 SQLite data model and loader

16 Test coverage and verification

17 Current data snapshot and worked results

18 Known limits and recommended engineering work

19 Glossary and formula reference

20 Source file index

This guide describes the code and local data present in the repository on 21 October 2026. It distinguishes implemented behavior from recommended future improvements.

01 Purpose and operating boundaries

The Commodity Risk Platform is a local research application for turning historical prices into aligned asset returns, portfolio P&L, historical risk estimates, rolling VaR backtests, attribution tables, and deterministic stress scenarios. Its main user experience is a Streamlit dashboard, with Python modules for repeatable analysis and CSV exports.

The default example portfolio uses six illustrative USD notionals: Brent proxy 500,000; WTI proxy 300,000; natural gas proxy 200,000; coffee proxy 100,000; Bitcoin 100,000; and Ethereum 50,000. Gross notional is 1,250,000 USD and net notional is 1,250,000 USD because every default position is long.

> The dashboard itself warns that positions and stress shocks are hypothetical. The calculations are historical estimates and do not guarantee a maximum loss, forecast future returns, or represent live holdings.

Who this guide is for

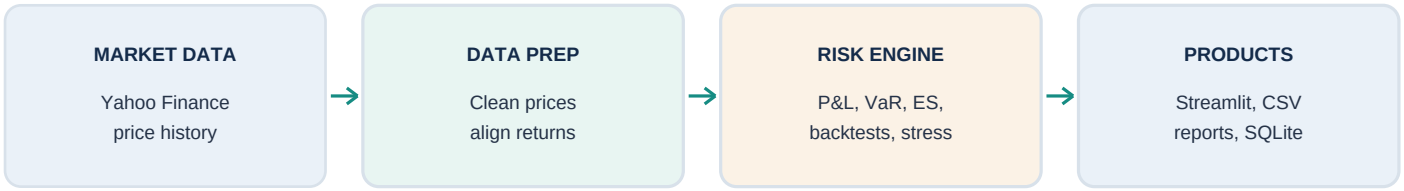
- A user who needs to understand what each dashboard control changes.
- A developer who needs to locate data ingestion, calculations, UI, exports, or tests.
- A reviewer who needs to reproduce the current 95% and 99% VaR backtest results.

What the system does not currently claim

- It does not stream live prices or live positions into the dashboard.
- It does not model intraday market risk, funding, margin, liquidity, transaction costs, or portfolio rebalancing.
- It does not provide production risk governance, model approval, data lineage, or audited risk reports.
- It is not an investment recommendation or a guarantee of future risk.

02 End-to-end architecture

The application is organized as a local batch pipeline. Raw vendor downloads are normalized into per-instrument close files, then converted into an aligned daily return panel. The risk engine reads that panel and supplies the dashboard and report scripts. A SQLite path exists for optional persistence of commodity prices and returns.



Each stage writes files used by the next stage; database loading is optional.

Stage	Input	Main work	Output
Acquire	Yahoo Finance tickers	Download unadjusted OHLCV history from 2018-01-01	data/raw/*.csv
Clean	Vendor CSV files	Select Close, parse dates, remove invalid or duplicate rows	data/processed/<asset>.csv
Align	Six clean price series	Compute returns and align crypto to the commodity calendar	data/processed/returns/aligned_returns.csv
Analyze	Aligned returns and notionals	P&L, VaR/ES, attribution, correlations, stress, backtests	Dashboard and reports/*.csv

Stage	Input	Main work	Output
Persist (optional)	Processed prices and returns	Create schema and upsert four commodity series	database/commodity_risk.db

The pipeline is not one automatic command. Run each stage in order when upstream data changes. The dashboard does not download prices; it reads the aligned return CSV already on disk.

03 Repository map

Path	Role
app/pages/dashboard.py	Streamlit application: portfolio setup, metrics, charts, diagnostics, exports.
src/risk/risk_engine.py	Data loader, portfolio P&L, historical VaR/ES, attribution, correlations, stress, report generation.
src/risk/backtesting.py	Rolling historical VaR threshold generation and breach flags.
src/risk/backtest_summary.py	Breach summary, Kupiec POF, Christoffersen independence, conditional coverage.
src/risk/run_backtest.py	Batch backtest at a fixed 99% confidence and 250-observation window.
src/risk/analyze_breaches.py	Reads backtest CSV, finds breach clusters, exports breach detail.
src/data/download_prices.py	Canonical downloader for four commodity proxies and two crypto assets.
src/data/clean_prices.py	Normalizes raw Yahoo Finance CSV formats into positive close-price rows.
src/analytics/returns.py	Builds the shared valuation-calendar return panel.
src/data/load_database.py	Creates SQLite tables and upserts four commodity price and return series.
src/data/verify_database.py	Prints SQLite table names, row counts, and instrument names.
database/schema.sql	Relational schema for instruments, prices, returns, positions, metrics, and ingestion logs.
tests/	Unit coverage for risk engine functions, rolling backtest behavior, and diagnostics.
data/	Local raw and processed CSVs; most raw/intermediate files are ignored by Git.
reports/	Generated CSV outputs for risk metrics, backtests, diagnostics, attribution, and stress.
requirements.txt	Unpinned runtime packages: numpy, pandas, plotly, streamlit.
download_prices.py	Untracked root-level downloader; currently differs from canonical src/data/download_prices.py.

No top-level README or populated docs directory was present in the inspected working tree. This PDF therefore serves as a first consolidated operator and developer guide.

04 Setup and runbook

Run commands from the repository root so relative paths such as reports/ and database/ resolve consistently. A local .venv is present in the current checkout. The repository does not pin package versions and requirements.txt does not currently list every package used by optional scripts or tests.

Suggested Windows PowerShell setup

```
cd C:\Users\Ashray Gupta\commodity-risk-platform
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pip install yfinance pytest
```

yfinance is imported by the price downloader; pytest is needed to run the test suite. They are not in the current requirements.txt. If the virtual environment is new, install those packages explicitly or add a maintained development requirements file before onboarding another machine.

Run the data and analytics stages

```
python -m src.data.download_prices
python -m src.data.clean_prices
python -m src.analytics.returns
python -m src.risk.risk_engine
python -m src.risk.run_backtest
python -m src.risk.analyze_breaches
```

The first three commands refresh inputs in sequence. risk_engine writes portfolio summary, attribution, correlation, and stress CSVs. run_backtest writes rolling VaR results and diagnostics. analyze_breaches depends on var_backtest_results.csv having already been generated.

Start the dashboard and optional database

```
streamlit run app/pages/dashboard.py
python -m src.data.load_database
python -m src.data.verify_database
python -m pytest -q
```

The loader resolves its database path relative to the source file. The verifier uses the relative path database/commodity_risk.db, so it should be invoked from the repository root.

05 Price acquisition

The canonical downloader is src/data/download_prices.py. It requests unadjusted Yahoo Finance data starting 2018-01-01, turns off the progress display and multithreaded downloads, and writes each returned frame to data/raw/.csv. Failures are printed and processing continues, which means a completed command does not by itself prove that every instrument downloaded.

Internal name	Yahoo Finance ticker	Class
brent_proxy	BZ=F	Commodity futures proxy
wti_proxy	CL=F	Commodity futures proxy
natural_gas_proxy	NG=F	Commodity futures proxy
coffee_proxy	KC=F	Soft commodity proxy
bitcoin	BTC-USD	Cryptocurrency
ethereum	ETH-USD	Cryptocurrency

The script imports yfinance, which is absent from the current requirements.txt. It does not archive vendor query parameters or an immutable source snapshot, so future downloads can change the local dataset.

There is also an untracked root file named download_prices.py. It downloads only the four commodity tickers and is separate from the six-asset canonical module. Prefer python -m src.data.download_prices after reviewing the root-level file before deciding whether to retain it.

06 Price cleaning and validation

src/data/clean_prices.py accepts two CSV layouts because yfinance can write a two-row column header or a conventional one-row header. It first tries the two-row format and selects a column whose first header level is Close. If that is unavailable or parsing fails, it retries with a single-row header and looks for a Close column.

Check or transformation	Behavior
Date parsing	First column or index is parsed as UTC-aware timestamps, then serialized as YYYY-MM-DD.

Check or transformation	Behavior
Close conversion	Close values are coerced to numbers; invalid dates and prices are dropped.
Positive price rule	Rows with close less than or equal to zero are removed.
Duplicates	Duplicate date and instrument pairs keep the last row.
Ordering	Clean output is sorted by date.
Empty data	The instrument fails with a ValueError if no valid rows remain.
Missing raw file	The instrument is skipped with a printed message; the overall script continues.

Per-instrument clean files are written under data/processed/. They are ignored by Git in the current .gitignore, as are raw prices and processed panels other than aligned_returns.csv. Re-running clean_prices can therefore change local analytics without changing the visible Git diff.

07 Return construction and calendar alignment

src/analytics/returns.py reads the four commodity price files and two crypto price files. It checks each individual series for duplicate dates and non-positive or non-numeric closes before calculating simple percentage returns.

Step	Exact current approach	Consequence
Commodity calendar	Inner join the four commodity price indexes.	Only dates shared by all four commodity series remain.
Commodity returns	pct_change(fill_method=None) on the shared price panel.	No implicit price fill is used for commodity returns.
Crypto calendar	Join BTC and ETH prices, then forward-fill onto commodity valuation dates.	Crypto is marked at each commodity valuation date.
Crypto returns	pct_change(fill_method=None) after the price reindex.	Crypto changes between valuation dates are recorded as one interval.
Final panel	Concatenate commodity and crypto returns, sort, then drop rows with any missing value.	Every final row has all six return fields.
Files	Write aligned_returns.csv and individual_returns.csv.	The current implementation writes the same aligned panel to both files.

This makes a common valuation calendar, but it does not create seven-day portfolio P&L for crypto. Weekend crypto moves are incorporated at the next commodity valuation date, so daily timing differs from a calendar that marks crypto every day. The dashboard calls this out as a data caveat.

load_returns reads data/processed/returns/aligned_returns.csv, coerces every column to numeric, drops rows containing any missing field, rejects an empty file and duplicate column names, and returns a date-sorted frame.

08 Portfolio exposure and P&L

risk_engine.py stores the six default notionals in POSITIONS. A positive notional represents a long exposure; a negative notional represents a short exposure. The dashboard exposes one signed USD number input per instrument, so a user can change size and direction.

For a return r(i,t) and signed notional N(i), daily instrument P&L is N(i) multiplied by r(i,t). Portfolio P&L is the sum across configured instruments for each date. The code does not compound holdings, rebalance notionals, or model financing.

In symbols: `P(t) = sum_i N(i) * r(i,t)`. If an instrument is missing from the aligned frame, the engine raises a ValueError and identifies the missing column names.

Instrument	Default signed USD notional	Gross contribution
Brent proxy	$500,000	$500,000
WTI proxy	$300,000	$300,000
Natural gas proxy	$200,000	$200,000
Coffee proxy	$100,000	$100,000
Bitcoin	$100,000	$100,000
Ethereum	$50,000	$50,000
Total	$1,250,000 net	$1,250,000 gross

09 Historical VaR and Expected Shortfall

The portfolio risk engine converts P&L into losses by multiplying P&L by -1. Historical VaR is the empirical confidence quantile of those losses. The implementation floors the reported VaR at zero, so it never reports a negative loss threshold.

Historical VaR: `max(0, quantile_c(-P))`, where c is the selected confidence and P is the P&L series.

Historical Expected Shortfall: find the VaR threshold, select losses greater than or equal to it, average those losses, then floor the result at zero. This is a discrete empirical tail average with inclusive ties.

historical_var_es requires a confidence strictly between 0 and 1 and at least 100 valid P&L observations. It returns confidence, observation count, VaR in USD, and Expected Shortfall in USD.

Confidence	Full-sample VaR	Full-sample Expected Shortfall	Observation count
95%	$39,813.53	$62,460.43	2,212
99%	$73,037.60	$106,806.65	2,212

These are full-sample estimates for the default portfolio. They are distinct from the rolling backtest thresholds, which use only the previous lookback observations at each date.

10 Rolling VaR backtest

historical_var_backtest sorts valid P&L by date and calculates a new threshold for each out-of-sample observation. With window W, the current observation is tested against P&L history from positions i-W through i-1. The current observation is not included in its own threshold window.

For confidence c, the loss threshold at time t is `max(0, -quantile_(1-c)(P[t-W:t]))`. A breach is defined strictly as `actual_PnL(t) < -VaR_loss(t)`. A loss exactly equal to the threshold is not counted as a breach.

Input or rule	Current default and validation
Window	250 observations in the dashboard and batch backtest; dashboard allows 30 to 1,000, stepping by 10.
Confidence	Must be strictly between 0 and 1; dashboard offers 95% or 99%.
Minimum data	The number of non-missing sorted observations must be greater than the window.
Output fields	Date index, actual_pnl_usd, var_loss_usd, breach.
Expected row count	T - W; 2,212 input rows minus 250 equals 1,962 backtest rows.

This is an expanding out-of-sample sequence with a fixed-width rolling estimation window. It is historical simulation rather than a forecast. Results depend on the selected portfolio notionals, the aligned data, quantile interpolation, confidence, and lookback.

11 Confidence controls and the 95% versus 99% issue

The dashboard has two separate confidence controls. Their names are similar, but they affect different calculations. The overview also displays both 95% and 99% Historical VaR simultaneously, regardless of either selector.

Control	Changes	Does not change
Portfolio VaR confidence	Selects the confidence used by the Expected Shortfall card.	The overview's two VaR cards remain fixed at 95% and 99%.
VaR backtest confidence	Rolling thresholds, breach flags, breach summary, expected breach rate, Kupiec test, independence test, conditional coverage test, and plotted threshold label.	Full-sample VaR and ES cards in the overview.

The backtest selector is now explicitly keyed as backtest_confidence and its help text states that it affects rolling thresholds and diagnostics. The portfolio selector has its own stable key and explains that the overview shows both VaR levels. This should prevent changing one control and expecting the other section to change.

Both selectors default to 99%. If the screen seems unchanged, confirm which selector was changed and inspect the VaR backtesting tab. The portfolio overview intentionally always shows the 95% and 99% VaR metrics next to each other.

The standalone command-line run_backtest.py is separate from the dashboard control. It has a constant CONFIDENCE_LEVEL = 0.99 and LOOKBACK_WINDOW = 250. Its CSV files will remain 99% until the script configuration is changed; the dashboard selector does not modify those files.

12 Statistical backtesting tests

The backtest produces hit indicators. The summary module compares the observed hit sequence with the selected confidence and calculates three likelihood-ratio diagnostics.

Test	Question	Implementation summary
Kupiec proportion of failures (POF)	Is the total breach frequency consistent with the expected rate q = 1-c?	Compares Bernoulli likelihoods under fixed expected q and observed breach rate; LR statistic is evaluated against chi-square with 1 degree of freedom.
Christoffersen independence	Are breaches independent over time, rather than more likely after a previous breach?	Counts adjacent hit transitions n00, n01, n10, n11 and compares independent Bernoulli likelihood with a first-order Markov alternative; chi-square with 1 degree of freedom.
Conditional coverage	Does the hit sequence have both the expected frequency and independence?	Adds the Kupiec and independence LR statistics; computes the upper-tail p-value for chi-square with 2 degrees of freedom.

A p-value below 0.05 is flagged as a rejection at the 5% level. A non-rejection is not proof that a model is correct. These are asymptotic tests and their practical power depends on sample size and breach structure.

The dashboard gives priority to the independence or conditional-coverage result: if either rejects, it warns the user to investigate breach clustering and calibration. If neither rejects but Kupiec does, it gives a frequency-only warning. Otherwise it says the displayed tests did not reject and cautions that this does not establish model adequacy.

The code rejects empty result frames, invalid confidence, missing or non-binary breach values, fewer than two observations for independence, and sequences that cannot satisfy its transition-count preconditions.

13 Dashboard walkthrough

Launch with streamlit run app/pages/dashboard.py from the project root. The app reads aligned_returns.csv on each script run, validates configured instruments, calculates portfolio P&L and diagnostics, then renders five tabs.

Tab	Visible content	Main interpretation
Portfolio overview	Asset count, gross and net notional, daily P&L volatility, 95% and 99% historical VaR, selected-confidence ES, daily P&L chart, cumulative P&L chart.	A high-level view of static-notional historical portfolio risk.
VaR backtesting	Confidence and lookback caption, observations, breaches, observed and expected breach rates, actual P&L versus rolling threshold, statistical test table, warning or info message, breach details, CSV download.	Whether the historical hit frequency and sequence are consistent with the selected VaR confidence.
Asset attribution	Standalone risk table, gross-notional chart, 99% standalone VaR chart, historical return correlations, CSV download.	Per-asset summaries and linear correlations; not a marginal contribution or covariance attribution model.
Stress testing	Three named scenarios, portfolio and per-instrument impacts, charts, table, CSV download.	Deterministic shock arithmetic under stated hypothetical shocks.
Data and exports	Aligned row count, instrument count, missing cell count, date range, loaded instruments, last 20 returns, CSV download, crypto calendar caveat.	Quick check of the exact return panel consumed by analytics.

All portfolio positions are sidebar number inputs. Position changes feed P&L, risk estimates, attribution, and stress outputs. The backtest lookback and confidence feed the backtest calculations. The portfolio confidence control is used for the ES metric, while the overview's 95% and 99% VaR cards are always independently computed.

The P&L chart is a sum of estimated daily P&L at constant notionals, not a rebalanced portfolio return series. The cumulative chart sums those daily P&Ls from the start of the aligned sample.

14 Command-line reports and output files

File	Creator	Contents
reports/portfolio_risk_summary.csv	python -m src.risk.risk_engine	Full-sample 95% and 99% VaR/ES, counts, portfolio notionals and volatility.
reports/asset_risk_attribution.csv	python -m src.risk.risk_engine	Standalone metrics for each configured asset.
reports/correlation_matrix.csv	python -m src.risk.risk_engine	Pairwise return correlations.
reports/stress_scenarios.csv	python -m src.risk.risk_engine	Per-instrument P&L and portfolio totals under each scenario.
reports/var_backtest_results.csv	python -m src.risk.run_backtest	One row per out-of-sample date, including P&L, VaR loss and breach.
reports/var_backtest_diagnostics.csv	python -m src.risk.run_backtest	Summary, Kupiec, independence, and conditional-coverage outputs.
reports/var_breach_analysis.csv	python -m src.risk.analyze_breaches	Breach dates, actual P&L, VaR and excess loss.

Dashboard downloads

- dashboard_var_backtest_results.csv: selected dashboard confidence and lookback for the current positions.

- dashboard_asset_risk_attribution.csv: current positions and per-asset attribution metrics.

- dashboard_stress_scenarios.csv: instrument-level stress results.

- dashboard_aligned_returns.csv: the return panel loaded by the application.

A generated dashboard download is an in-memory browser response. It is not automatically the same as a tracked reports/ file. The batch backtest has a fixed confidence and should not be mistaken for the current dashboard selection.

15 SQLite data model and loader

database/schema.sql defines six tables. src/data/load_database.py creates the local database, enables foreign keys, executes the schema, inserts or updates instrument metadata, and uses SQLite conflict handling to upsert price and return records.

Table	Key fields and purpose	Current loader behavior
instruments	Unique symbol, display name, asset class, data source.	Upserts four commodity proxy definitions.
prices	Instrument id, date, close price; composite primary key and positive close check.	Upserts processed prices for the four commodities.
daily_returns	Instrument id, date, daily return; composite primary key.	Upserts four commodity return series from individual_returns.csv.
portfolio_positions	Portfolio name and signed USD notional per instrument.	Schema exists; current loader does not populate it.
risk_metrics	Calculation date, portfolio, metric, confidence, value, methodology.	Schema exists; current loader does not populate it.
ingestion_log	Source file, UTC time, row count, status, message.	Schema exists; current loader does not write ingestion records.

The schema can represent crypto and multiple portfolios, but the current loader's INSTRUMENTS mapping includes only Brent, WTI, natural gas, and coffee. Database persistence is optional and is not used by the dashboard, which reads the CSV return panel directly.

verify_database.py prints the table list, counts for selected tables, and instrument names. It does not assert expected counts or validate numerical integrity, so treat it as a visibility utility rather than a comprehensive database test.

16 Test coverage and verification

Run the suite from the root with python -m pytest -q. At the latest test run in this work session, all 24 tests passed. pytest is currently not listed in requirements.txt.

Test file	Coverage
tests/test_risk_engine.py	Portfolio P&L arithmetic, missing instrument failure, correlation symmetry, and a deterministic stress scenario.
tests/test_backtesting.py	Prior-only windows, confidence effect, insufficient history, invalid confidence, breach summary, data validation, Kupiec behavior, dependence behavior, and conditional coverage composition.
tests/test_backtest_summary.py	Summary statistics, transition counts, p-value bounds, conditional coverage, empty/missing inputs, and minimum sequence length.

A new regression test checks that a higher confidence produces no narrower rolling loss thresholds and changes breach counts on a deterministic synthetic series. The Streamlit page was also changed to provide distinct selector keys and explanatory help text.

The automated Streamlit AppTest smoke attempt did not complete within the tool run window and was stopped. The unit suite passes, but the full browser-rendered Streamlit interaction has not been confirmed in this session. Use the manual run command in section 04 to inspect the page and switch the backtest confidence.

17 Current data snapshot and worked results

The aligned data used for this snapshot contains six series and 2,212 observations from 3 January 2018 through 21 October 2026. Results use the default positions and a 250-observation rolling window. Values are specific to this local data snapshot and will change when prices or notionals change.

Metric	95%	99%
Full-sample Historical VaR	$39,813.53	$73,037.60
Full-sample Expected Shortfall	$62,460.43	$106,806.65
Rolling backtest observations	1,962	1,962
Rolling breaches	120	24
Observed breach rate	6.1162%	1.2232%
Expected breach rate	5.0000%	1.0000%
Kupiec POF p-value	0.02814	0.33693
Independence p-value	0.000293	0.002452
Conditional coverage p-value	0.000128	0.006414

At 95%, the Kupiec test rejects the expected breach frequency at the 5% level, while the independence and conditional-coverage tests also reject. At 99%, the Kupiec test does not reject breach frequency, but independence and conditional coverage do reject. The dashboard therefore warns under both confidence settings because its warning rule prioritizes the dependence and conditional-coverage results.

These values independently confirm that the rolling confidence selector changes the model thresholds and hit count. The 99% threshold is higher because the loss quantile is taken deeper into the tail; on the observed data that produces fewer exceedances.

18 Known limits and recommended engineering work

The following are code-grounded limitations or next steps. They are not claims that the current calculations are unusable; they define where the implementation stops and what would improve reproducibility or production readiness.

Area	Current limit	Suggested next increment
Dependency management	No versions are pinned; yfinance and pytest are missing from requirements.txt.	Separate runtime and development dependencies, pin a tested range, and record Python version.
Data lineage	Raw and per-instrument cleaned files are Git-ignored; vendor downloads can change.	Record source ticker, retrieval time, file hash, date range, and cleaning diagnostics in an ingestion manifest.
Return calendar	Crypto is marked only on the commodity valuation calendar.	Make valuation calendar configurable; report skipped or aggregated crypto intervals.
Portfolio modeling	Daily P&L uses constant USD notionals and simple returns.	Document rebalancing, currency, contract multipliers, roll methodology, financing, fees, and missing dates before adding realism.
VaR methodology	Empirical historical quantiles with a zero floor; no volatility scaling or parametric benchmark.	Add method labels and compare historical, filtered historical, and parametric models only with reviewable assumptions.
Backtest interpretation	Likelihood-ratio p-values are asymptotic and not a proof of adequacy.	Add report explanations, confidence intervals, sample-size context, and optionally robust small-sample analysis.
Database completeness	Loader stores only commodity price/return series; positions, risk metrics, and ingestion_log are unused.	Either complete audited persistence or keep CSV as the explicit source of truth and document that choice.
Batch/UI parity	The batch backtest is hard-coded at 99% and 250 observations.	Accept CLI options or factor a shared config so batch and dashboard outputs can match.
Testing	Unit tests do not currently automate a full Streamlit browser session.	Add a fast AppTest that asserts both selector values and key metrics for a tiny fixture.
Duplicate script	Untracked root download_prices.py differs from the canonical six-ticker module.	Review provenance, then remove, rename, or merge it intentionally.

One especially important semantic distinction is that historical VaR is a threshold for a specified horizon under the sample, not a promise that losses will stay within that amount. A 99% daily VaR targets about a 1% hit probability only under the model's assumptions; dependence and changing distributions can make realized sequences behave differently.

19 Glossary and formula reference

Term	Meaning in this project
Aligned return	A simple percentage change for an asset on the shared valuation-date index.
P&L	Estimated USD profit or loss: signed notional multiplied by asset return, summed across assets.
Confidence c	Tail quantile level used for a VaR estimate; 95% corresponds to 5% nominal exceedance, 99% to 1%.
VaR	A positive loss threshold estimated from a historical loss quantile; not a maximum-loss limit.
Expected Shortfall	Average observed loss at or beyond the VaR threshold, using the implementation's inclusive tail rule.
Breach	An out-of-sample P&L strictly below the negative VaR loss threshold.
POF	Proportion-of-failures test of expected versus observed breach frequency.
Independence	Test of whether current breach probability depends on the previous breach state.
Conditional coverage	Joint frequency and independence likelihood-ratio test.
Notional	Signed USD exposure used as a simple multiplier of return; not a fully specified futures contract quantity.

Formulas

Simple return: `r(t) = close(t) / close(t-1) - 1`.

Portfolio daily P&L: `P(t) = sum_i N(i) * r(i,t)`.

Historical VaR: `max(0, quantile_c(-P))`.

Expected Shortfall: `max(0, mean(losses where loss >= VaR))`.

Rolling hit: `P(t) < -max(0, -quantile_(1-c)(P[t-W:t]))`.

Nominal expected hits in n observations: `n * (1-c)`; actual rates can differ due to finite samples, model fit, ties, or dependence.

20 Source file index

This guide is based on the files below. Paths are relative to the repository root, C:/Users/Ashray Gupta/commodity-risk-platform.

File	What to inspect there
app/pages/dashboard.py	Controls, metric wiring, chart fields, diagnostic warnings, and downloads.
src/risk/risk_engine.py	Positions, data loader, portfolio calculations, stress scenarios, analytics exports.
src/risk/backtesting.py	Rolling threshold and breach calculation.
src/risk/backtest_summary.py	Summary and likelihood-ratio diagnostics.
src/risk/run_backtest.py	Batch backtest config and exported diagnostics.
src/risk/analyze_breaches.py	Consecutive breach analysis and CSV export.
src/data/download_prices.py	Six Yahoo Finance ticker definitions and download settings.
src/data/clean_prices.py	CSV parsing, row cleanup and per-instrument output.

File	What to inspect there
src/analytics/returns.py	Commodity intersection calendar and crypto reindex/forward-fill.
src/data/load_database.py	SQLite upsert logic and current four-commodity coverage.
src/data/verify_database.py	Database inspection output.
database/schema.sql	Table constraints and relationships.
tests/test_risk_engine.py	Core P&L and stress behavior.
tests/test_backtesting.py	Rolling backtest and statistical behavior; includes confidence regression.
tests/test_backtest_summary.py	Summary and diagnostic edge cases.
requirements.txt and .gitignore	Declared packages and tracked-data boundaries.

End of guide. The most direct next development steps are to make dependencies reproducible, automate the dashboard selector interaction, unify batch and dashboard backtest configuration, and clarify the valuation calendar and data lineage.

Post-snapshot update

This addendum records repository changes made after the 13-page guide baseline dated 21 October 2026. Where an earlier section describes fixed batch settings or missing packages, use the updated details below.

What changed

Area	Current behavior
Dashboard label	The sidebar now calls the first control **Expected Shortfall confidence**. It changes the Expected Shortfall card; the overview still shows both 95% and 99% Historical VaR cards.
Batch backtest	The runner accepts `--confidence 0.95` or `--confidence 0.99` and `--window N`. Defaults remain 99% and 250 observations. Example: `python -m src.risk.run_backtest --confidence 0.95 --window 250`.
Shared portfolio	The batch runner now imports the default positions from `src.risk.risk_engine`, keeping its six instruments and notionals aligned with the dashboard. Reports resolve relative to the project folder.
Dependencies	`requirements.txt` now includes `yfinance`. `requirements-dev.txt` includes the runtime requirements and `pytest` for development checks.
Streamlit sizing	Dashboard charts and tables now use `width="stretch"`, replacing the deprecated `use_container_width` argument reported by the installed Streamlit version.

Verification and open work

All 26 automated tests pass. The live browser check confirmed the selector updates backtest metrics at the 250-observation lookback: 95% produced 1,962 observations, 120 breaches, and a 6.12% observed rate against 5.00% expected; 99% produced 1,962 observations, 24 breaches, and a 1.22% observed rate against 1.00% expected. The default 99% view was restored. The dashboard rendered after the sizing update with no browser console warnings; Streamlit AppTest itself did not finish within the tool's 30-second run window.

Remaining engineering work includes package version pinning, source-data lineage, the crypto valuation calendar, broader database persistence, and a reliable automated AppTest run. The separate untracked root-level `download_prices.py` remains outside the commits; review it before deciding whether to merge or remove it.

This addendum supplements the code-grounded guide and supersedes earlier statements about the batch runner's fixed settings, missing yfinance/pytest requirements, and unverified browser behavior. Historical risk estimates remain illustrative and depend on local data and selected assumptions.